

Sommario esecutivo

Vogliamo costruire un prototipo di gioco tattico a turni simultanei ispirato ad *Atlas Reactor* usando Unreal Engine. Il progetto funge da percorso di apprendimento: definiamo prima le **meccaniche core** da replicare (fasi di turno, azioni simultanee, tipologie di abilità, movimento su griglia, coperture, UI, ecc.), poi suddividiamo lo sviluppo in tre ambiti di progetto (prototipo minimo, medio, completo) con tempi ed obiettivi di studio crescenti. Mappiamo le competenze UE da acquisire (Blueprint, C++, gameplay framework, rete, IA, animazioni, UI, input, fisica) e costruiamo un **curriculum didattico progressivo**. Ogni lezione avrà obiettivi chiari, prerequisiti, durata stimata, esercizi pratici, deliverable (Blueprint/C++ da consegnare) e risorse ufficiali. Per la transizione da C#, consigliamo di usare inizialmente i Blueprint (per prototipi rapidi) e poi integrare C++ “idiomatico UE” [46†L119-L127] [37†L133-L139]. Il plugin **UnrealCLR** (o similare) consente di eseguire C# in UE [28†L223-L227], ma ad oggi non ha supporto stabile UE5 [29†L233-L237], quindi è più sicuro imparare C++ UE nativo. Infine, definiamo la struttura del progetto (cartelle, asset pipeline, naming convenzioni) e le pratiche di version control (Git + LFS, .gitignore, lock file) [39†L37-L41] [39†L64-L72] [41†L410-L418] per uno sviluppo ordinato. Nel report approfondiremo ogni punto con dettaglio, tabelle di comparazione (scope di progetto, sequenza lezioni, roadmap), riferimenti alla documentazione ufficiale UE, esempi (es. progetto Lyra di Epic [46†L159-L162]) e risorse in italiano quando disponibili.

Meccaniche di gioco chiave

- **Fasi di turno simultanee:** come *Atlas Reactor*, ogni turno ha due modalità. In **Decision Mode** (20 secondi) ogni giocatore pianifica segretamente le proprie azioni [9†L143-L152]. Poi in **Resolution Mode** tutte le azioni si risolvono contemporaneamente in quattro sub-fasi sequenziali: *Prep* (trappole, buff/debuff), *Dash* (schivate/cariche con possibilità di danno), *Blast* (attacchi a bersaglio fermo, push-back) e *Move* (movimento finale, maggior spostamento per chi non ha usato abilità) [9†L143-L152]. In ogni fase le azioni di tutti i giocatori sono calcolate simultaneamente [9†L143-L152]. Non ci sono iniziative personali vere e proprie, ma si potrebbero aggiungere bonus di reattività come variante.
- **Tipi di azione:** ogni personaggio (o *Freelancer*) ha una serie di abilità uniche – di solito 3 abilità attive più un’abilità finale (Ultimate) e la possibilità di muoversi. Gli slot azione corrispondono a *Prep*, *Dash*, *Blast* e *Move* [9†L143-L152] [5†L197-L201]. Le abilità hanno costi e cooldown: per esempio la prima abilità di solito **non ha cooldown** ed è spamabile ogni turno, mentre la quinta (ultimate) consuma energia [5†L197-L201]. È importante progettare il sistema di cooldown/ricarica energia e magari status temporanei (stordito, avvelenato, ecc.).
- **Movimento e mappa:** il gioco si svolge su una mappa (ad es. griglia quadrata) con ostacoli. Ogni turno un personaggio può muoversi di un certo numero di caselle (più elevato se non usa abilità) [9†L143-L152]. Si può considerare un sistema di cover: ad esempio, in *Atlas Reactor* stare **dietro coperture** riduce il danno del 50% dalla direzione corrispondente [47†L205-L212] (ciò favorisce posizionamenti tattici). Le mappe possono essere arene chiuse (nessun fog of war tradizionale in *Atlas Reactor*, ma si può valutare un leggero limitare la line-of-sight dietro ostacoli).
- **Obiettivi di vittoria:** partite 4 vs 4 PvP con respawn continuo, vince il team che raggiunge 5 kill o ha più uccisioni dopo 20 turn [9†L139-L147]. Nel prototipo minimo si può semplificare a una singola uccisione o sopravvivenza, introducendo il conteggio kill e il respawn in fasi successive.

- **Interfaccia (UI):** serve un HUD che mostri vita/energia personaggi, icone abilità (cooldown), mini-mappa o indicazioni, timer di turno e pulsanti per scegliere azioni. Unreal supporta UI con UMG (Widget) 【32†L131-L137】. La UI deve guidare il giocatore nella scelta delle azioni simultanee e visualizzare i risultati di ogni fase.

In sintesi, il prototipo replicherà il loop *pianifica-muovi-e risolvi*, con gestione di azioni simultanee per fase 【9†L143-L152】, abilità con cooldown/energia 【5†L197-L201】, movimento tattico sulla griglia con coperture 【47†L205-L212】, e infine un sistema di punteggio (kill o obiettivi).

Ambiti di sviluppo (Scope)

Per evitare un progetto troppo grande, suddividiamo le funzionalità in livelli di complessità **crescenti**. Ciascun *scope* ha obiettivi di apprendimento specifici.

Scope	Funzionalità principali	Sforzo stimato	Obiettivi di apprendimento
MVP minimale	- Singolo personaggio per lato (1v1) o bot base - Griglia di movimento semplice - Una fase di turno (decision + risoluzione con due sub-fasi semplificate) - Una sola abilità per personaggio - UI basilare (un indicatore vita/cooldown)	~2-4 settimane	- Configurare UE5, primo Blueprint/C++ a schermo 【37†L119-L123】 - Capire il gioco a turni simultanei 【9†L143-L152】 - Uso base dei Blueprint (movimento, input) e logica di turno - Primi accenni a C++ (opzionale)
Funzionalità medie	- Più personaggi (2 vs 2, con ruoli diversi) - Mappa con coperture (danni ridotti) 【47†L205-L212】 - Abilità multiple (Prep, Dash, Blast, Move) con cooldown/energia 【5†L197-L201】 - Parte di UI (icone abilità, timer, scoreboard) - AI semplice (bot che sceglie mosse casuali)	~2-3 mesi	- Implementare sistema di abilità e cooldown 【5†L197-L201】 - Gestione multi-personaggio e spostamenti tattici - Realizzare UI più complessa (UMG HUD) 【32†L131-L137】 - Fondamenti di C++ (per es. classi base dei personaggi) e Blueprint avanzati - Nozioni di AI (Controller/Behavior Tree di base)
Completo (full)	- Team 4v4 e sistema multiplayer di rete (repliche e server) 【32†L53-L62】 - Varie classi/abilità avanzate e animazioni (AnimBP) - Mappe multiple con layout vari - Gestione partita (rete, lobby, sincronizzazione turno) - UI complete (menu, scoreboard, effetti visivi/sonori)	~5-6+ mesi (team)	- Networking Unreal: replication, RPC, GameMode/GameState 【32†L33-L41】 【32†L53-L62】 - Competenze di animazione (AnimBlueprint, blendspace) - Scripting avanzato C++ (sistemi di gioco) 【46†L119-L127】 - Ottimizzazione, rifinitura, testing approfondito

Nota: Seguiamo il consiglio di realizzare un **prototipo finito e piccolo** prima di espandere 【51†L260-L268】. Ad esempio, un MVP “arena survival” con le meccaniche base insegna molto (movimento, combattimento, UI, refactoring) 【51†L260-L268】. Secondo Rambod Dev, un progetto contenuto (es. un solo livello, meccanismo di sparo, win/lose chiari) è ben focalizzato ed insegnante 【51†L260-L268】.

In linea con queste dimensioni, il primo MVP sarà contenuto (uno o due personaggi, una mappa piccola, azioni base) 【51†L260-L268】. Man mano il progetto cresce, si allargano le caratteristiche (più abilità, bot, interfaccia dettagliata, multiplayer), e con esso la complessità (richiesta di competenze avanzate). I tempi stimati (es. 2-4 settimane per MVP, 2-3 mesi per scope medio, ecc.) sono indicativi e vanno adattati alla curva di apprendimento.

Competenze UE necessarie

Per implementare il gioco servono vari ambiti di conoscenza di UE:

- **Blueprint vs C++:** Inizialmente prototipa con Blueprint (flow visivo) per scene semplici 【37†L119-L123】. Epic raccomanda di usare C++ per i **sistemi core** (frenanti, AI, gestione avanzata) e Blueprint per definire comportamenti e iterare velocemente 【46†L119-L127】 【37†L125-L131】. Ad esempio, definire in C++ la logica del ciclo di turno e poi esporre funzioni a Blueprint 【46†L119-L127】.
- **Gameplay Framework:** servono classi base come *GameMode*, *GameState*, *GameInstance*, *PlayerController*, *Pawn/Character* 【32†L53-L62】 【32†L114-L122】. Ad esempio, il *GameMode* definisce le regole (es. condizioni di vittoria) e genera i giocatori 【32†L114-L122】; il *PlayerController* gestisce l’input umano e possiede un *Pawn* (il personaggio fisico) 【32†L53-L62】. Conoscere il rapporto *Controller-Pawn* è cruciale. Il *PlayerState* tiene dati di ciascun giocatore (HP, punteggio) mentre il *GameState* tiene stato globale (es. turni trascorsi). 【32†L53-L62】 【32†L114-L122】.
- **Input e possesso:** configurare l’Input Mapping (tasti/ mouse) e associare i controller ai *Pawn*. UE gestisce ciò tramite *PlayerController* e input events nei Blueprint/C++.
- **AI:** implementare un *AIController* che possiede un *Pawn* nemico e un *BehaviorTree* semplice (es. muovi casualmente, attacca). Epic fornisce *BehaviorTree* e *NavMesh*; l’AI Controller integra i sistemi di navigazione 【32†L53-L62】.
- **Animazioni:** animare i personaggi con *Animation Blueprints* (AnimBP). Usare *Skeleton*, *State Machine* per camminata, attacco, morte. Le risorse ufficiali coprono l’uso di AnimBP (per es. il template *ThirdPerson UE4*).
- **Interfaccia utente (UI):** creare HUD e menu con *UMG Widget Blueprint*. La documentazione ricorda che UI e HUD mostrano informazioni sovrapposizione 【32†L131-L137】. Ad esempio, una *HealthBar* o il timer di turno.
- **Fisica e collisioni:** basica (collidere personaggi e proiettili), Unreal fornisce componenti standard (collision primitive, movimento integrato).
- **Rete (Multiplayer):** per il scope avanzato, serve networking con attore di replica (*GameState*, *PlayerState*), RPC e sessioni online. Epic doc spiega come *GameMode/GameState* si replicano tra server e client 【32†L114-L122】. Si useranno anche *PlayerController* e *PlayerState* per ogni client 【32†L53-L62】.

Per ogni ambito si prevedono lezioni dedicate (es. tutorial ufficiali su “Gameplay Framework” 【32†L114-L122】), sulle animazioni, sull’input). Esempi di progetto (come *Lyra* di Epic) mostrano integrazioni Blueprint/C++ 【46†L159-L162】: utile studiare il loro codice di partenza. Nei prossimi paragrafi pianificheremo passo-passo l’apprendimento di queste competenze.

Curriculum didattico (lezione per lezione)

Ecco una sequenza esemplificativa di lezioni progressive. Ogni lezione indica: obiettivi di apprendimento, prerequisiti, durata stimata, esercizi pratici con deliverable (Blueprint/C++), risorse consigliate (documentazione, tutorial, esempi). Le prime lezioni usano i Blueprint, poi si introduce gradualmente il C++.

Lezione	Obiettivi	Prerequisiti	Durata	Esercizi/ Deliverable	Risorse consigliate
1. Introduzione a UE5	- Installare UE5 e configurare un progetto C++ (o Blueprint). - Familiarizzare con interfaccia UE (Viewport, Content Browser, World Outliner).	Conoscenza base programmazione (C#)	1–2 ore	Creare progetto UE5. Posizionare un paio di Static Mesh (cubi) nella scena.	Doc ufficiale “QuickStar UE5”; tutorial “Beginner UE5 Interface”; repository Lyra [46†L159-L162]
2. Primo Pawn e Input	- Creare un <i>Pawn</i> (o <i>Character</i>) Blueprint controllabile. - Gestire input base: movimento WASD, jump.	Lezione 1	2–3 ore	Blueprint di un personaggio semplice che si muove su assi X/Y.	Tutorial UE5 “Movement” (Epic), doc “Character Setup” (UG3)
3. Sistema di turno – prototipo	- Implementare il flusso di turno simultaneo: fase Decision e simple Resolution. - Usare un timer (20s) in Blueprint.	Lezione 2	3–4 ore	Blueprint: funzionalità di <i>presa decisione</i> (lock input) e <i>risoluzione</i> (mostra azioni su schermo).	Articolo “Blueprints per turni simultanei” (esercizio DIY), repo GitHub es. Lyra, Pagina Epic “State Machine”.
4. Movimento e griglia	- Implementare movimento a griglia: selezione caselle raggiungibili. - Calcolo percorsi basico (node grid).	Lezione 3	4–6 ore	Blueprint/C++: mostri figura che si sposta per caselle (fade cubi evidenziati).	Tutorial UE “Grid-Based Movement”; doc UE Navigation.

Lezione	Obiettivi	Prerequisiti	Durata	Esercizi/ Deliverable	Risorse consigliate
5. Azioni (Prep/Dash/Blast)	- Implementare almeno una abilità in ciascuna categoria: es. un "Dash" offensivo e un "Blast" ad area. - Gestire animazioni o effetti semplici.	Lezione 4	6-8 ore	Creare due abilità in Blueprint con cooldown, una muove e una danneggia.	Tutorial UE "Ability System" introduttivo; documentazione "GameplayAbilities" (se disponibile).
6. Sistemi di abilità e cooldown	- Strutturare il gestore di abilità: livelli, cooldown, costi (energia). - UI element base: iconcina e timer.	Lezione 5	4-6 ore	Blueprint: creare sistema "CooldownManager" con variabili e timeline.	Epic doc "Gameplay Ability System" (Epic Online Services), video "Abilities in UE5".
7. Turno completo e risoluzione	- Orchestrare le azioni in ordine di fase: tutte le azioni "Prep" vengono eseguite, poi le "Dash", ecc. - Mostrare effetti in sequenza, conflitti (chi muore).	Lezione 6	4-6 ore	Blueprint/C++: estendere sistema di turno per gestire 4 fasi di risoluzione.	Articolo "Four-phase turn system in UE" (blog di sviluppo indie).
8. Più personaggi e coperture	- Aggiungere un secondo personaggio controllabile o bot (AI semplice). - Implementare coperture statiche: riduzione danni quando personaggio alle spalle (50% [47+L205-L212]).	Lezione 7	6-8 ore	Nuovo livello: due personaggi, mappa con muri. Blueprint: logica riduzione danni in copertura.	Doc UE "Collision and Cover"; tutorial Blueprint su cover system.

Lezione	Obiettivi	Prerequisiti	Durata	Esercizi/ Deliverable	Risorse consigliate
9. UI avanzata	- Realizzare HUD: health bar, energie, icone abilità. Timer di turno. Schermata menu start.	Lezione 8	4-6 ore	Creare Widget Blueprint con barre, icone, testo dinamico.	Guida UMG su Unreal Docs; sample Blueprint Lyra HUD 【46+L159-L162】 .
10. Integrazione C++ base	- Creare una classe C++ <code>AGameMode</code> personalizzata per gestire regole (turni, vittoria). - Esposizione di funzioni a Blueprint (UFUNCTION).	Lezione 9	6-8 ore	Scrivere in C++ logica base di avanzamento turno e vittoria, richiamata da Blueprint.	Tutorial “GameMode in C++” (Epic), doc “UFUNCTION BlueprintCallable”.
11. Bot AI avanzata	- Sviluppare un <i>AIController</i> più sofisticato: behavior tree che sceglie mosse (move, abilità) seguendo semplici regole. - Uso di NavMesh per spostamento.	Lezione 10	6-8 ore	Creare Behavior Tree con patrolling e attacco. C++/Blueprint per decisioni AI.	Tutorial UE “AI Behavior Tree”; Sample C++ AI Controller.
12. Multigiocatore (Networking)	- Prototipare il multiplayer: GameSession/ PlayerState replicati, possesso di Pawn remoto, sincronizzazione turni tra client.	Lezione 11	8-12 ore	Estendere GameMode/State a C++ con proprietà replicate. Test su LAN con 2 client.	Documentazione Epic “Networking (UE)”, esempio “Shooter Game” su GitHub.

Lezione	Obiettivi	Prerequisiti	Durata	Esercizi/ Deliverable	Risorse consigliate
13. Animazioni e polished	- Aggiungere animazioni al personaggio (AnimBP per camminata, attacco). - Rifinire effetti visivi/sonori, bilanciamento finale.	Lezione 12	4-6 ore	Implementare uno scheletro e AnimBP; aggiungere suoni base (colpi).	Tutorial "Animazione UE4/5", sito Unacademy.it (ITA); esempi Lyra.

Ciascuna risorsa consigliata sopra è tipicamente trovabile nella documentazione ufficiale Epic o nel forum, oppure in tutorial di qualità (YouTube, Epic Community). Ad esempio, Epic fornisce tutorial sul **Gameplay Framework** (classes Actor, Pawn, Controller) [32†L53-L62] [32†L114-L122] e sulla **combinazione Blueprint/C++** [46†L119-L127]. Il progetto di esempio *Lyra* (scaricabile dal Launcher) illustra molte tecniche di gioco multiplayer e integrazione C++/Blueprint [46†L159-L162]. In italiano, articoli e corsi di sviluppo Unreal (es. tutorials di GameCodeSchool) possono integrare le risorse ufficiali.

Percorso da C# a UE5

Con fondo C#, le opzioni per sviluppare in Unreal sono: **solo Blueprint**, **plugin .NET (UnrealCLR/UnrealSharp)**, o **C++ nativo**. I pro e contro:

- **Blueprint only**: accessibile subito (specie per chi non conosce C++), ottimo per prototipi rapidi [37†L119-L123] [37†L125-L131]. Tuttavia, le prestazioni in massimi progetti e la complessità possono soffrire. Il coding rimane visivo e "aperto" (ma complessi Blueprint possono diventare poco leggibili).
- **UnrealCLR/UnrealSharp**: plugin open-source che integrano un .NET runtime in UE, consentendo di scrivere script in C# [28†L223-L227]. UnrealCLR "integra nativamente il .NET host in UE col Common Language Runtime" [28†L223-L227]. In teoria permette di usare C# completo. **Attenzione**: allo stato attuale (UE5/2026) UnrealCLR non è una soluzione "ufficiale" e non ci sono release stabili per UE5 [29†L233-L237]. Alcuni sviluppatori segnalano che il progetto è stagnante e potenzialmente incompatibile con UE5 [29†L233-L237]. UnrealSharp è un'alternativa simile. In pratica, queste soluzioni sono più adatte a prototipi personali; non godono della stessa documentazione/assistenza di C++.
- **Switch to C++ UE**: la strada più "idiomatica" secondo Epic è imparare C++ in UE. Epic suggerisce di usare C++ per definire i sistemi di base e Blueprint per estendere il comportamento [46†L119-L127]. In C++ i concetti di C# si traducono facilmente: classi derivate da Actor, UPROPERTY, garbage collection managed da Unreal. Il codice C++ è testuale (facilmente versionabile) e offre prestazioni più elevate [46†L62-L70] [46†L88-L97]. Le principali sfide sono la sintassi UE (macro UCLASS/UPROPERTY/UFUNCTION) e la gestione della memoria UE, ma ci sono tutorial per passare da Blueprint a C++ [46†L119-L127].
Esercizi consigliati: convertire un Blueprint semplice in C++ (usando la funzionalità "Convert to C++" di UE) e studiare il codice generato. Seguire la guida Epic "*Coding in UE: Blueprint vs C++*" [46†L119-L127] e esempi UE di esposizione C++/Blueprint. Ad esempio, creare in C++ una classe base `APlayerCharacter` con UFUNCTION esposti a Blueprint, così da implementare la logica principale in C++ e usarla nei Blueprint per animazioni e feedback. In sintesi, per ora si

consiglia di sfruttare i Blueprint per velocità, imparando nel frattempo le basi di C++ UE tramite piccole classi (con Rider o Visual Studio). Solo in casi avanzati opteremo per UnrealCLR.

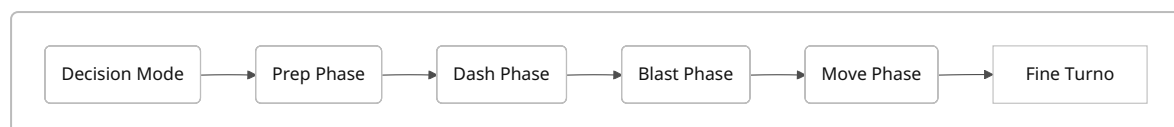
Struttura del progetto, asset pipeline e version control

- **Struttura cartelle:** definire sin dall'inizio un'organizzazione coerente del *Content*. Ad esempio, cartelle separate per "Blueprints" (classi gioco), "Maps" (livelli), "Characters" (mesh/anim), "UI" (widget), "Gameplay" (C++ classes), "Materials", ecc. Unreal usa percorsi fissi per referenziare asset, quindi rinominare file/cartelle può generare *redirectors* e rompere i riferimenti [41†L410-L418]. È consigliato adottare un *progetto guideline* statico e rifattorizzare nelle pause (weekend) [41†L426-L428].
- **Pipeline asset:** Modelli 3D (FBX), texture e materiali importati in UE. Seguire best practice (pivot centrati, livello di dettaglio - LOD - per mesh). Creare collisioni semplificate negli static mesh. Per le animazioni, usare Skeleton condivisi se possibile. Documentarsi su "*Asset workflow*" in UE: ossia, import-in-Engine -> regolare fisica/materiali -> salvare nell'Engine.
- **Version Control (Git+LFS):** usare un repository Git (es. GitHub/GitLab) con **Git LFS** per file binari (mesh, texture) [39†L37-L41]. Creare un `.gitignore` che escluda cartelle derivate (DerivedDataCache, Binaries, Intermediate) [39†L37-L41]. Anchorpoint (guida UE) consiglia proprio di abilitare Git LFS per tutti gli asset grandi [39†L37-L41]. Un VCS è fondamentale per *backup*, sincronizzazione e lavoro in team [39†L64-L72]. Permette di fare frequenti *commit* (ad ogni task concluso) [39†L64-L72], mantenere uno storico, effettuare branching (feature branches) e code review (pull request). Consigli pratici:
 - Committare almeno una volta al giorno e sempre al termine di un piccolo compito [39†L64-L72]. Aggiungere messaggi descrittivi (eventualmente con riferimento a ticket).
 - Locking: se più persone modificano asset importati (Blueprint, livello, mesh), usare le funzionalità di *file lock* (LFS) per evitare conflitti [39†L64-L72] [41†L410-L418]. In UE5 esiste anche un *Revision Control Plugin* che interagisce con Perforce/Git (opzionale).
 - Branching: usare un branch "main" stabile e creare branch su funzionalità (es. "feature/input", "feature/ability-system"). Unire (merge) i branch una volta testati (pull request). Per piccoli team singoli, si può lavorare direttamente su main ma con commit strutturati.
 - Redirector: dopo aver spostato asset, *Click destro* → *Fix up Redirectors* in UE per eliminare i puntatori creati [41†L410-L418]. Ciò evita il diff clutter.
 - Progetto Git: versionare SOLO la cartella principale del progetto (oltre alle sottocartelle Content e Source). Non caricare Binaries/Intermediate/DerivedData; usare `.gitignore` standard UE (disponibile in rete) e LFS. Anchorpoint offre esempi di config Git per UE [39†L37-L41].

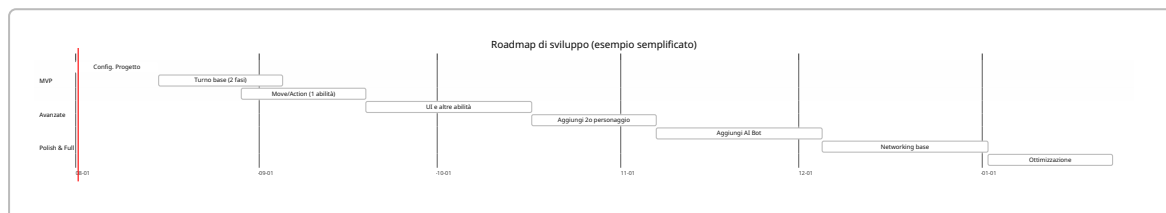
Un adeguato controllo versione è indispensabile per un progetto prolungato. Come sottolinea Anchorpoint, permette *evitare perdite di lavoro e collaborare meglio* (backup, sincronizzazione, lock di file) [39†L64-L72]. Ad esempio, si potranno coordinare su Git l'implementazione del sistema di turno e la creazione del sistema di abilità in parallelo. Inoltre, la struttura del repository deve riflettere l'architettura del gioco (separando logica di gioco, asset di livello e UI) per facilità di manutenzione.

Diagrammi di flusso e roadmap

Di seguito alcuni diagrammi per chiarire la sequenza delle fasi di turno e una roadmap di sviluppo (linea temporale).



Suddivisione del turno: in ogni fase i giocatori eseguono simultaneamente le azioni pianificate (trappole/buff, schivate, attacchi, spostamento) 【9†L143-L152】 .



Roadmap di massima: le attività di sviluppo vengono raggruppate per fase. MVP (colore verde) implementa il loop base, poi si aggiungono funzionalità medie (blu) e infine networking e rifiniture (viola). Le durate sono stime indicative.

Conclusioni

Questo progetto garantisce un apprendimento **graduale e strutturato** di Unreal Engine partendo da conoscenze C# preesistenti. Si parte da un prototipo giocabile ridotto, si arricchisce passo passo con nuove meccaniche e competenze tecniche, fino ad arrivare eventualmente a un gioco completo. Cogliendo spunti dalla documentazione UE (Gameplay Framework 【32†L53-L62】 【32†L114-L122】 , AI, Networking) e dai sample ufficiali (es. *Lyra* 【46†L159-L162】) si avrà un percorso solido. Le tabelle sopra delineano fasi e milestone chiare; i riferimenti citati offrono approfondimenti e best practice. Con dedizione e iterazione rapida (grazie ai Blueprint) si potrà padroneggiare progressivamente C++ e i sistemi di UE necessari. L'approccio passo-passo massimizzerà l'apprendimento (come sottolineato dai consigli su scope ridotti 【51†L260-L268】) e consentirà di creare un gioco tattico simultaneo ben progettato, pur partendo da zero in C++.

Fonti principali: Documentazione Epic UE5 (Gameplay Framework 【32†L53-L62】 【32†L131-L137】 , Blueprints vs C++ 【46†L119-L127】), wiki *Atlas Reactor* (gioco di riferimento) 【9†L139-L147】 【9†L143-L152】 【5†L197-L201】 【47†L205-L212】 , risorse educative (itat. itamde 【37†L119-L123】 【37†L125-L131】) e guide sul version control in Unreal 【39†L37-L41】 【39†L64-L72】 【41†L410-L418】 . Questi forniscono esempi concreti e linee guida per ogni aspetto del progetto.